AI ASSISTED CODING – STUDENT ASSIGNMENT

CODE REFACTORING: IMPROVING LEGACY CODE WITH AI SUGGESTIONS

5 Hands-on Tasks

Student Name	Shital kumari	**Roll No.**	2503a520r2
Batch / Section	67	**Date**	13-10-2026
Course	AI Assisted Coding	**Faculty**	

Learning Objective

Understand code refactoring and use AI suggestions to improve readability, maintainability, structure, and efficiency of legacy Python code without changing its intended behavior.

Instructions for All Tasks

- Understand the original code before asking AI to modify it.
- Use an AI coding assistant to suggest improvements.
- Compare the original and refactored versions.
- Run both versions and verify that the behavior/output remains correct.
- Record at least one AI suggestion accepted and one suggestion rejected or modified.
- Do not submit AI-generated code without testing and understanding it.

TASK 1 – Improve Readability

Code / Problem

```
Legacy code:
a=80
b=70
c=90
x=(a+b+c)/3
if x>=50:
    print("pass")
else:
    print("fail")
```

Student Requirements

1. Ask AI to use meaningful variable names.
2. Create a function such as calculate_average().
3. Improve formatting and readability.
4. Run both versions and verify the same result.
5. Explain the improvements.

Submit: Original Code

a = 80

```
b = 70

c = 90

x = (a + b + c) / 3

if x >= 50:

print("pass")

else: print("fail")
```[cite: 1]

→ **AI Prompt**

Please refactor this Python code to use meaningful variable names, encapsulate the logic inside a function named `calculate_average()`, and improve overall formatting and readability while maintaining the exact same behavior."*[cite: 1]

AI Suggestion

Accepted Suggestion:Rename single-letter variables (`a`, `b`, `c`, `x`) to descriptive names (`score1`, `score2`, `score3`, `average_score`) and wrap the calculation in a function[cite: 1].

Rejected Suggestion: Using `math.fsum()` or external libraries (rejected because it adds unnecessary imports for a basic arithmetic operation).

→ **Refactored Code**

```
def calculate_average(score1, score2, score3):

    """Calculates the average of three scores and returns the result."""

      return (score1 + score2 + score3) / 3

def check_pass_fail(average_score, passing_threshold=50):

    """Determines pass or fail status based on the threshold."""

      if average_score >= passing_threshold:

       return "pass"

           return "fail"

if __name__ == "__main__":

score1, score2, score3 = 80, 70, 90

avg = calculate_average(score1, score2, score3)

result = check_pass_fail(avg)

print(result)
```[cite: ]

→ **Test Output**

```text pass

```[cite: 1]

→ **Explanation.**

The original code used cryptic single-letter variable names (`a`, `b`, `c`, `x`) which made the context unclear[cite: 1]. By introducing descriptive variable names and encapsulating the logic into `calculate_average()`, the code becomes modular, reusable, and much easier for other developers to read and maintain[cite: 1].

TASK 2 – Remove Repeated Code

Code / Problem

```
Legacy code:
name1 = "Arun"
marks1 = 85
if marks1 >= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")

name2 = "Priya"
marks2 = 72
if marks2 >= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

Student Requirements

6. Ask AI to identify duplicated logic.
7. Refactor the repeated logic into a reusable function.
8. Test the function with at least four students.
9. Explain why removing duplication improves maintainability.
10. Verify that output remains correct.

Submit: **Original Code**

```
name1 = "Arun"
marks1 = 85
if marks1 >= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")

name2 = "Priya"
marks2 = 72
if marks2 >= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

→ **AI Prompt**

Identify the duplicated logic in this code and refactor it into a reusable function so we can evaluate multiple students without rewriting the if-else block every time."*[cite: 1]

→ **Refactored Code**

```
def evaluate_student(name, marks, passing_mark=50):

"""Evaluates a student's marks and prints their status."""

status = "Pass" if marks >= passing_mark else "Fail"

 print(f"{name} {status}")

# Test the function with at least four students

if __name__ == "__main__":

 students_data = [ ("Arun", 85), ("Priya", 72), ("Rahul", 40), ("Meena", 95) ]

 for student_name, student_marks in students_data:

    evaluate_student(student_name, student_marks)
```

→ **AI Suggestion**

Accepted Suggestion: Create a reusable function `evaluate_student(name, marks)` that encapsulates the pass/fail print logic[cite: 1].

Rejected Suggestion: Storing all students in an external CSV file (rejected because it overcomplicates a simple script refactoring task).

→ **Test Output**

Arun Pass

Priya Pass

Rahul Fail

Meena Pass

→ **Explanation.**

Removing duplication follows the DRY (Don't Repeat Yourself) principle. Instead of duplicating the `if-else` print block for every new student, a reusable function allows us to scale cleanly, drastically improving maintainability and reducing the risk of copy-paste errors.

TASK 3 – Refactor a Long Conditional

Code / Problem

```
Legacy code:
age = 68
temperature = 39

if age >= 60 and temperature >= 38:
    print("High Priority")
elif age >= 60 and temperature < 38:
    print("Medium Priority")
elif age < 60 and temperature >= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

Student Requirements

11. Ask AI to simplify the conditional logic.
12. Create determine_priority().
13. Test at least four age/temperature combinations.
14. Compare original and refactored logic.
15. Explain whether AI preserved the original behavior.

Submit: Original Code

```
age = 68
temperature = 39

if age >= 60 and temperature >= 38:
    print("High Priority")
elif age >= 60 and temperature < 38:
    print("Medium Priority")
elif age < 60 and temperature >= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

→ **AI Prompt**

"Simplify this conditional logic using a function named `determine_priority()`. Check whether any duplicate outcomes (like Medium Priority) can be combined or streamlined using logical operators."

→ **AI Suggestion**

Accepted Suggestion:Combine the two `Medium Priority` conditions using logical operators or streamline the branching structure inside a clean function.

Rejected Suggestion:Replacing conditionals with a complex nested dictionary lookup (rejected as it reduces readability for simple numeric threshold checks).

→ **Refactored Code**

```
def determine_priority(age, temperature):
  """Determines patient or case priority based on age and temperature."""
if age >= 60 and temperature >= 38:
   return "High Priority"
elif age >= 60 or temperature >= 38:
  return "Medium Priority"
else: return "Normal Priority"
if __name__ == "__main__":
test_cases = [
 (68, 39), # Senior + High Temp -> High Priority
 (65, 37), # Senior + Normal Temp -> Medium Priority
(25, 39), # Young + High Temp -> Medium Priority
 (30, 36.5) # Young + Normal Temp -> Normal Priority ]
for a, t in test_cases:
print(f"Age: {a}, Temp: {t} -> {determine_priority(a, t)}")
```

→ **Test Output**

```
Age: 68, Temp: 39 -> High Priority
 Age: 65, Temp: 37 -> Medium Priority
 Age: 25, Temp: 39 -> Medium Priority
Age: 30, Temp: 36.5 -> Normal Priority
```

→ **Explanation.**

The AI successfully preserved the original behavior while shortening the conditional branches[cite: 1]. Because both `(age >= 60 and temperature < 38)` and `(age < 60 and temperature >= 38)` result in `"Medium Priority"`, they can be combined into `age >= 60 or temperature >= 38` (since the high-priority case is already caught first). This makes the code shorter, cleaner, and less prone to logic gaps.

TASK 4 – Refactor Inefficient List Processing

Code / Problem

```
Legacy code:
patient_ids = [101, 205, 309, 412, 518, 623]

target = 412
found = False
```

```
for patient_id in patient_ids:
    if patient_id == target:
        found = True

if found:
    print("Patient Found")
else:
    print("Patient Not Found")
```

Student Requirements

16. Ask AI to improve readability.
17. Ask whether a set or dictionary is better for frequent lookups.
18. Implement the justified suggestion.
19. Test existing and missing IDs.
20. Compare typical list and set lookup complexity.

Submit: Original Code

```
patient_ids = [101, 205, 309, 412, 518, 623]

target = 412
found = False

for patient_id in patient_ids:
    if patient_id == target:
        found = True

if found:
    print("Patient Found")
else:
    print("Patient Not Found")
```

→ **AI Prompt**

"Improve the readability of this lookup code. Also, advise whether a set or dictionary is better for frequent lookups compared to a list, and implement the recommended structure."

→ **AI Suggestion**

Accepted Suggestion:Convert the list into a Python `set` (`patient_ids_set`) to optimize lookups from O(n) to O(1) average time complexity, and use the native `in` keyword.

Rejected Suggestion:Using a binary search (rejected because the list is unsorted and sorting just for a lookup is inefficient unless sorted beforehand).

→ **Refactored Code**

def check_patient_exists(patient_set, target_id):

"""Checks if a patient ID exists using O(1) set lookup."""

if target_id in patient_set:

return "Patient Found"

return "Patient Not Found" # Test existing and missing IDs

if __name__ == "__main__":

Using a set for efficient frequent lookups

patient_ids_set = {101, 205, 309, 412, 518, 623}

print("Target 412:", check_patient_exists(patient_ids_set, 412))

print("Target 999:", check_patient_exists(patient_ids_set, 999))

→ **Test Output**

Target 412: Patient Found

Target 999: Patient Not Found

→ **Explanation**.

Complexity Comparison:A list lookup checks elements sequentially, resulting in O(n) time complexity (where n is the number of items). A Python `set` uses a hash table behind the scenes, providing an average time complexity of O(1) for membership tests (`in`). For frequent lookups in large collections, sets are significantly faster and cleaner.

TASK 5 – Complete Legacy Code Review with AI

Code / Problem

```
Legacy code:
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

total = 0
for s in students:
    total = total + s[1]

avg = total / len(students)

for s in students:
    if s[1] >= 50:
        print(s[0], "Pass")
    else:
        print(s[0], "Fail")

print("Average =", avg)
```

Student Requirements

21. Ask AI to identify at least five improvements.
22. Refactor using meaningful names and reusable functions.
23. Use suitable Python features when they improve clarity.
24. Run both versions and verify equivalent results.
25. Record AI suggestions as accepted/rejected with reasons.
26. Identify the most useful refactoring change and justify it.

Submit: Original Code

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

total = 0
for s in students:
    total = total + s[1]

avg = total / len(students)

for s in students:
    if s[1] >= 50:
        print(s[0], "Pass")
    else:
        print(s[0], "Fail")

print("Average =", avg
```

AI Prompt

"Review this legacy student record code. Identify at least five improvements, refactor it using meaningful variable names, separate concerns into reusable functions, and use appropriate Python built-ins."

→ **AI Suggestion**

1.Accepted: Use built-in functions like `sum()` instead of a manual accumulator loop.

2. Accepted:Unpack tuples cleanly in loops (`for name, score in students`) instead of indexing via `s[0]` and `s[1]`.

3. Accepted: Encapsulate average calculation and status printing into distinct functions.

4. Accepted: Use formatted strings (`f-strings`) for cleaner output formatting.

5. Rejected: Using Pandas DataFrames for processing 4 records (rejected because it adds an unnecessary heavy external dependency for a small list).

→ **Refactored Code**

```
def calculate_class_average(students):
  """Calculates the average score of the student class using built-in sum()."""
 if not students:
   return 0
 total_score = sum(score for name, score in students)
 return total_score / len(students)
def display_student_results(students, passing_threshold=50):
  """Displays pass/fail status for each student."""
 for name, score in students:
```

```
status = "Pass"  if score >= passing_threshold else "Fail"

 print(f"{name} - {status}")

# Execution and Testing

 if __name__ == "__main__":

student_records = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]
display_student_results(student_records)

avg_score = calculate_class_average(student_records)

print(f"Average = {avg_score}")
```

→ **Test Output**

```
Arun - Pass

Priya - Fail

Rahul - Pass

Meena - Pass

Average = 65.75
```

→ **Explanation.**

Replacing the manual total accumulator loop with Python's built-in `sum()` function combined with tuple unpacking (`for name, score in students`).